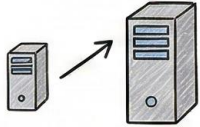


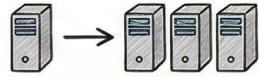
SYSTEM DESIGN FUNDAMENTALS

CORE PRINCIPLES

Scalability



Vertical - Scale Up



Horizontal - Scale Out

Handle growth (More Power vs. More Machines)

Reliability & Availability



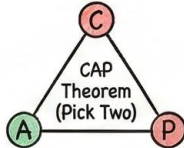
Fault Tolerance, Uptime
(e.g., 99.99%)

Performance



Low Latency,
High Throughput

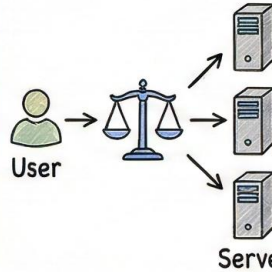
Consistency (CAP)



Data uniformity
across nodes.

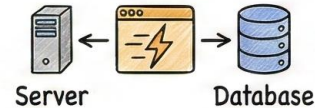
KEY BUILDING BLOCKS

Load Balancer



Distributes Traffic

Caching



Fast Data Access
(Temporary Storage)

Databases



SQL

Persistent Storage
(Structured/Flexible)



NoSQL

Message Queue



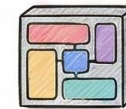
Producer

Consumer

Asynchronous comms
(Decoupling)

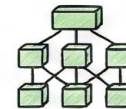
PATTERNS & TRADE-OFFS

Monolithic vs. Microservices



Monolith

Single unit, simple start



Microservices

Independent, scalable, complex



Trade-offs:

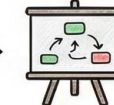
Design is about balancing
competing goals
(e.g., CAP)

Design Process

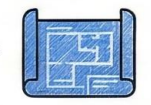
Understand -> Plan -> Refine



Requirements



High-Level
Design

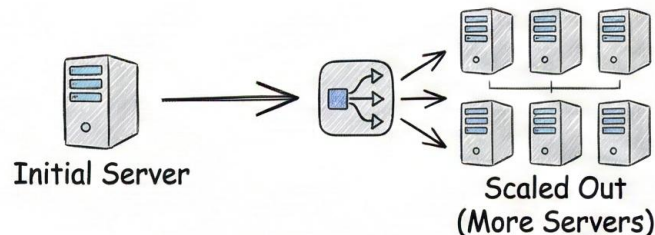


Detailed Design

GOAL: Build scalable, reliable, and performant systems that meet business needs.

HORIZONTAL VS VERTICAL SCALING (System Design)

HORIZONTAL SCALING (Scaling Out)



Add more machines to the pool.

Pros:



Elasticity
(Easy to add/remove)



Fault Tolerance
(No single point of failure)



Cost-Effective at Scale
(Commodity hardware)

Cons:



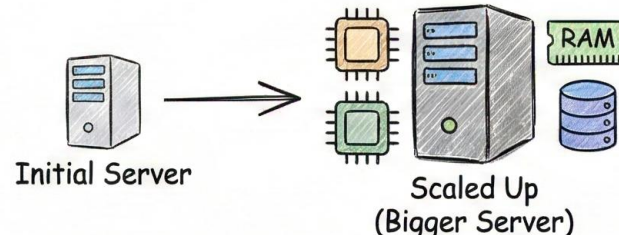
Complexity
(Requires load balancing, distributed systems)



Network Overhead
(Latency between nodes)

Best for: Massive scale, high availability, distributed applications
(e.g., Web/App Servers, NoSQL DBs).

VERTICAL SCALING (Scaling Up)



Add more power (CPU, RAM) to existing machine.

Pros:



Simplicity
(Easier to manage, no load balancer)



No Network Overhead
(Communication within one machine)



Lower Initial Cost
(For small scale)

Cons:



Limited Scalability
(Hardware limits)



Single Point of Failure
(If it goes down, everything stops)



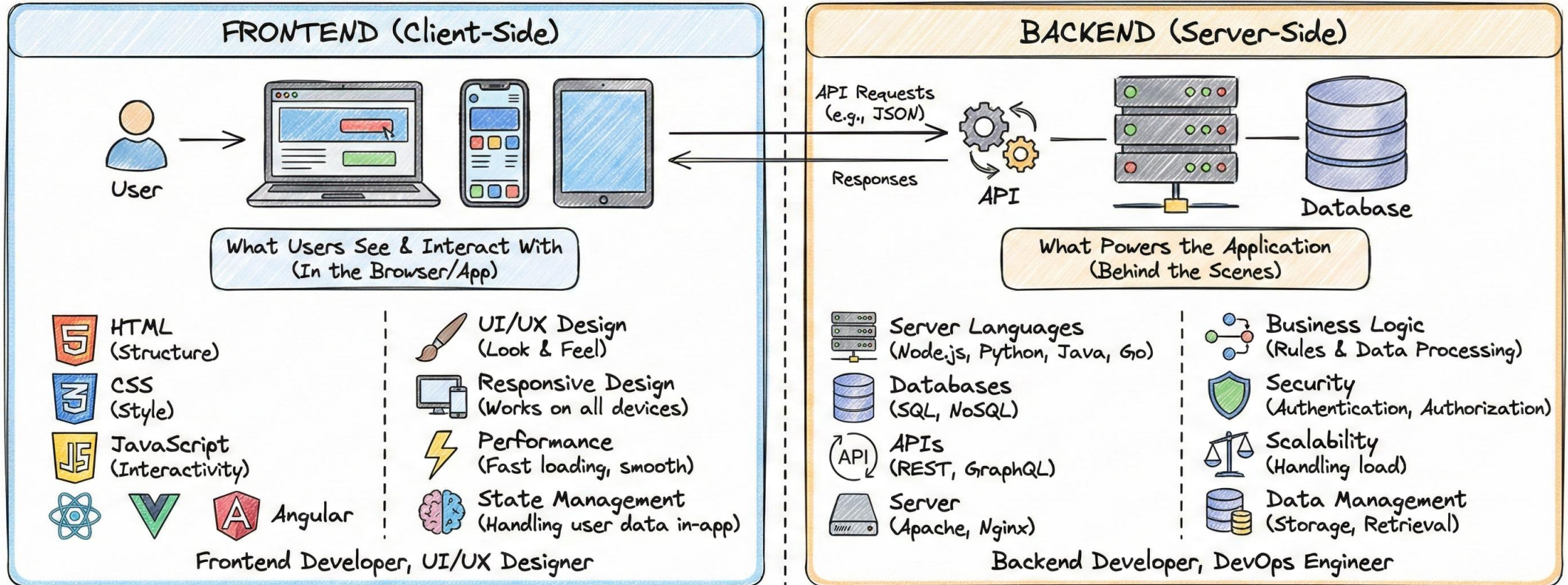
Downtime for Upgrades
(Requires restart)

Best for: Simplicity, initial growth, monolithic applications
(e.g., Traditional RDBMS, small-to-medium workloads).

KEY TAKEAWAY:

- Horizontal = More Machines (Elastic, Complex)
- Vertical = Bigger Machine (Simple, Limited)
- Choose based on scalability needs, budget, and application architecture.

FRONTEND VS BACKEND (System Design)



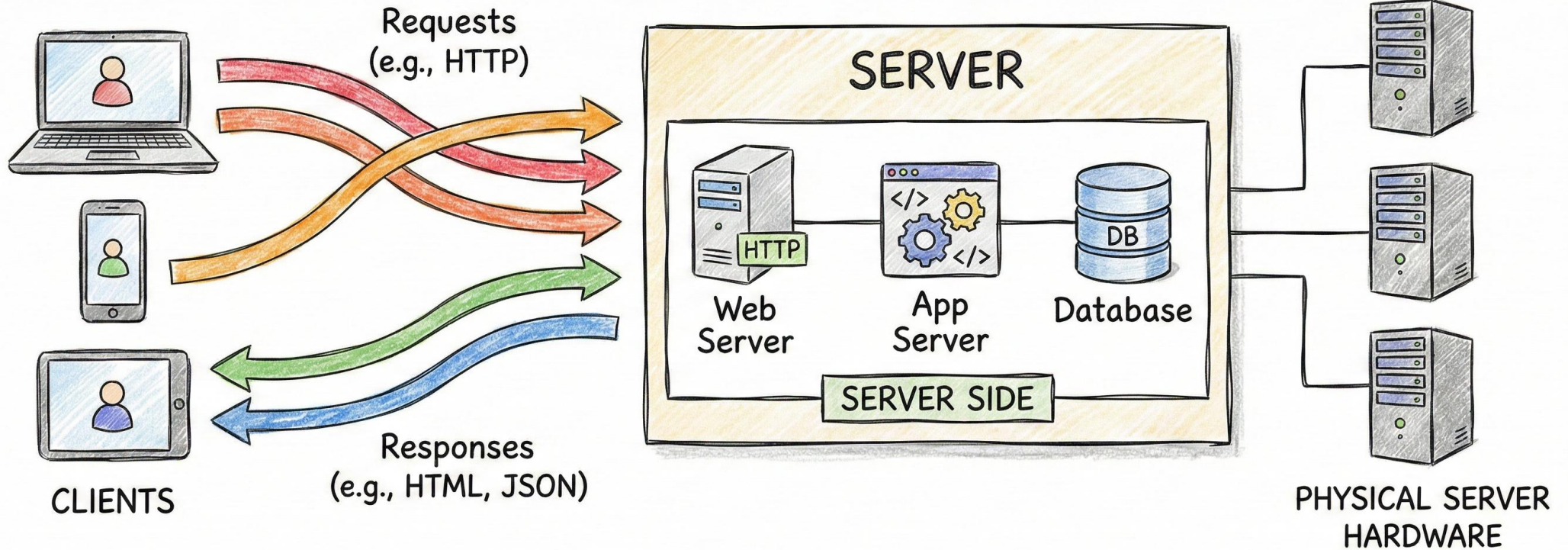
KEY DIFFERENCES & COLLABORATION:

Aspect
Focus
Execution
Data
Security

Frontend	Backend
User Experience	Logic & Data
Client's Device	Server/Cloud
Display & Input	Storage & Processing
Client-Side (Limited)	Server-Side (Critical)

They communicate via APIs to create a complete application.

CLIENT SERVER ARCHITECTURE (System Design)



KEY CHARACTERISTICS:

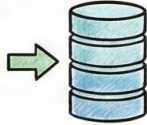
- Centralized Resources
- Client Initiates Requests
- Server Manages Data
- Scalable Server-Side

DATABASES - ADVANTAGES & CHALLENGES (System Design)

ADVANTAGES

Structured Data

ID	Name	Value
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____



Organized, easy to query & manage.

Data Integrity & Security



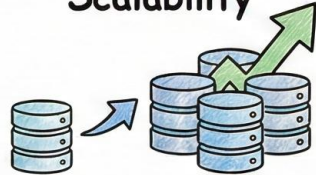
Ensures accuracy, consistency, and protection.

Efficient Retrieval



Quick access to specific information.

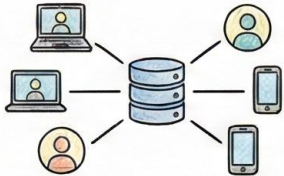
Scalability



Handles growing data volumes.

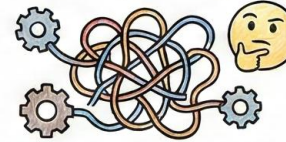
Concurrent Access

Multiple users can access simultaneously.



CHALLENGES

Complexity



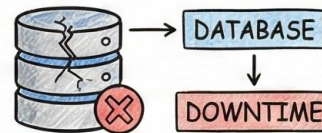
Difficult to design, set up & maintain.

Cost



Expensive hardware, software, and licensing.

Single Point of Failure



If it fails, the entire system may stop.

Performance Issues



Can become a bottleneck under high load.

Data Migration

Moving data can be complex and risky.

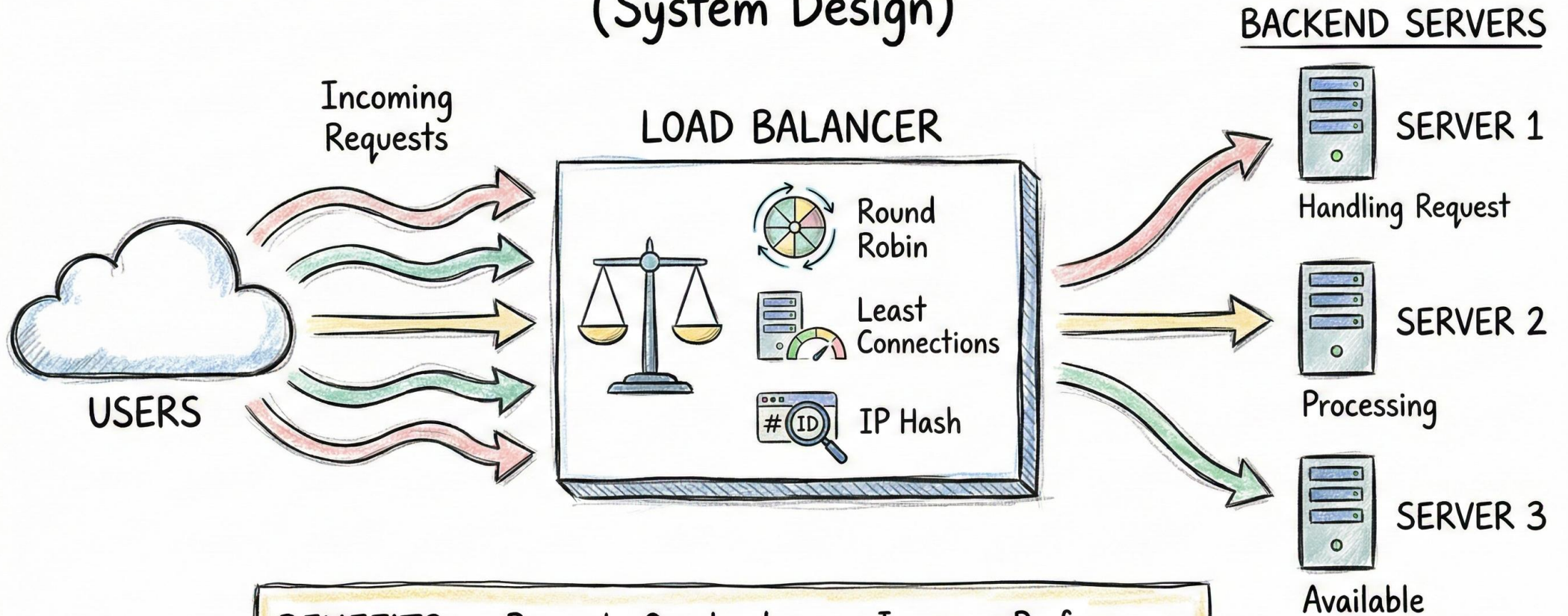


KEY TAKEAWAY:

- Balance Benefits & Trade-offs
- Choose the Right Type (SQL/NoSQL)
- Plan for Maintenance & Growth

HOW A LOAD BALANCER WORKS

(System Design)

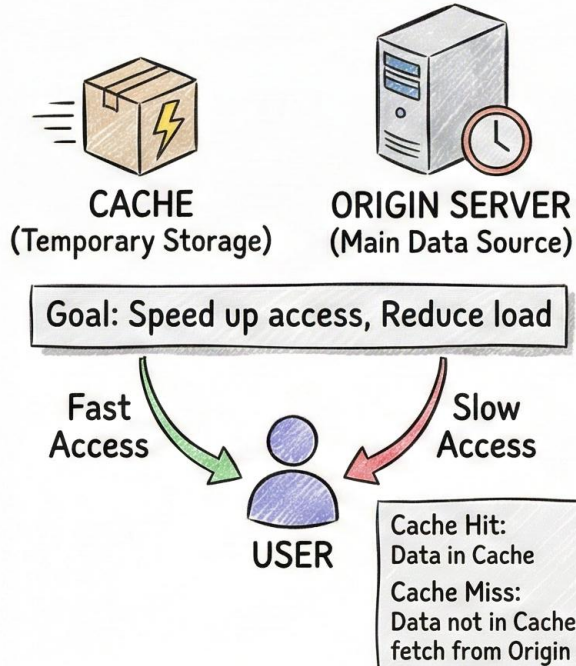


BENEFITS:

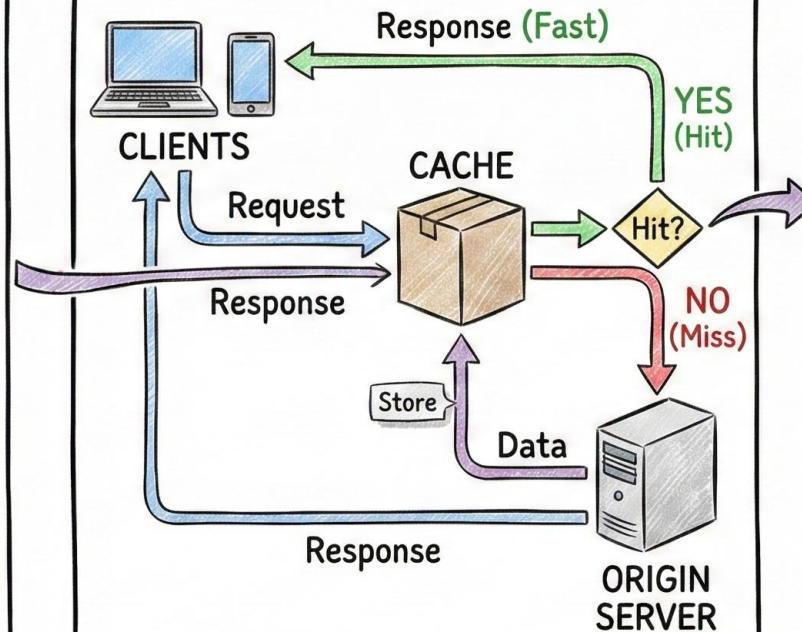
- Prevents Overload
- Improves Performance
- Increases Availability
- Scalability

CACHING - CONCEPTS & EVICTION TECHNIQUES (System Design)

CONCEPTS

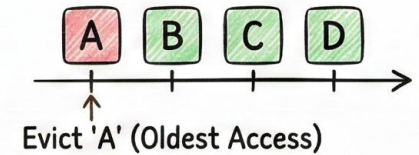


HOW IT WORKS

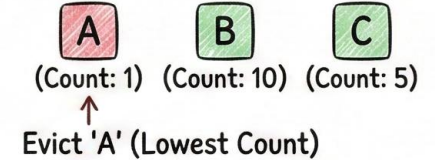


EVICTION TECHNIQUES

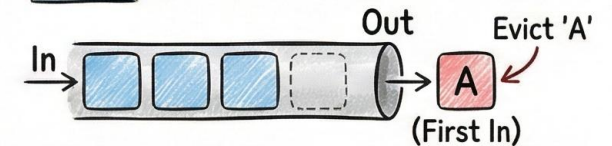
LRU (Least Recently Used)



LFU (Least Frequently Used)



FIFO (First-In, First-Out)

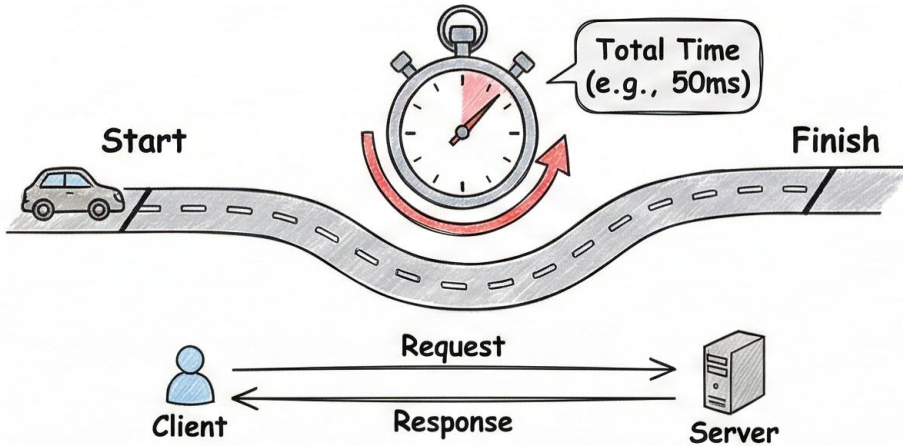


BENEFITS:

- Faster Response Times
- Reduced Server Load
- Lower Network Costs
- Improved Scalability

LATENCY VS THROUGHPUT (System Design)

LATENCY (Time to Complete One Task)



The time it takes for a single request to travel from client to server and back.

The time it takes for a single request to travel from client to server and back.



Think of it as:
The time for *one* car to travel from A to B.

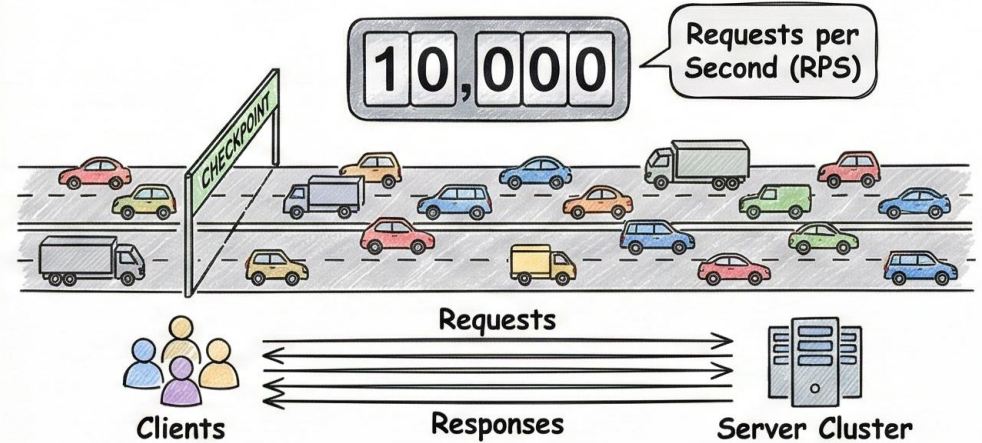


Measured in:
Milliseconds (ms) or
Seconds (s).



Goal: Minimize
(Lower is better).

THROUGHPUT (Tasks Completed Over Time)



The number of requests a system can handle in a given time period.



Think of it as: The
number of cars passing
a point in one hour.



Measured in:
Requests per Second (RPS),
Transactions per Second
(TPS), etc.



Goal: Maximize
(Higher is better).

KEY TAKEAWAY:

- Latency = Speed of *one* request (Time)
- Throughput = Volume of *many* requests (Rate)
- Both are crucial for system performance, but optimize based on needs.

AVAILABILITY & REDUNDANCY

(System Design)

AVAILABILITY (Uptime)



Uptime (e.g., 99.99%)

$$\text{Availability} = \frac{\text{Uptime}}{(\text{Uptime} + \text{Downtime})}$$

The "Nines"

99% (2 Nines)	→ 3.65 days downtime/year
99.9% (3 Nines)	→ 8.76 hours downtime/year
99.99% (4 Nines)	→ 52.6 mins downtime/year
99.999% (5 Nines)	→ 5.26 mins downtime/year



Downtime

REDUNDANCY (The Backup)



Primary
(Active)



Duplicate
components to
eliminate single
points of failure.

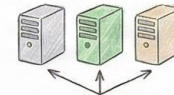
Replication



Secondary
(Standby/Backup)

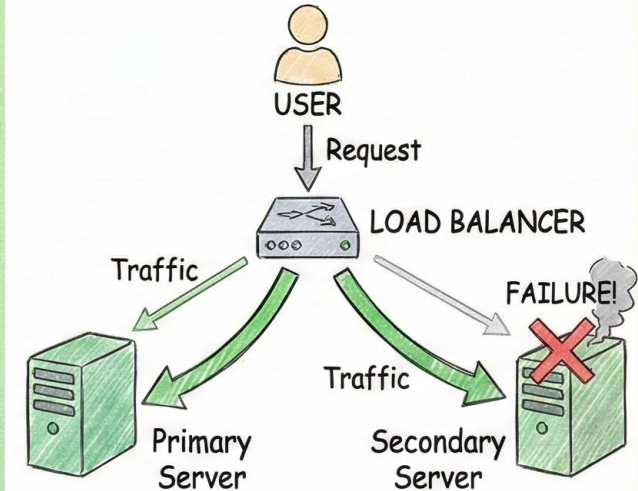


Active-Passive:
One active, one waits.



Active-Active:
Both active, sharing load.

HOW REDUNDANCY IMPROVES AVAILABILITY



Failover: Automatic switch to backup
component upon failure, minimizing downtime.

KEY BENEFITS & TRADE-OFFS:

Benefits:

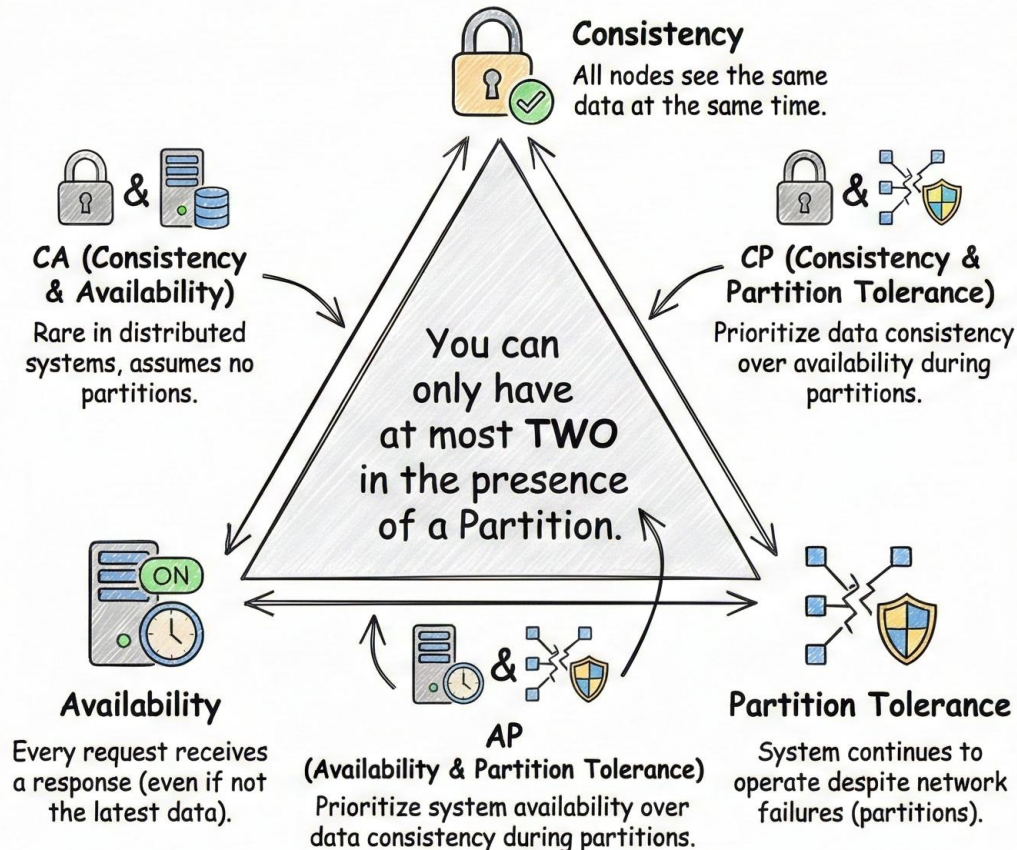
- Higher Uptime (Availability)
- Fault Tolerance
- Seamless Maintenance
- Disaster Recovery

Trade-offs:

- Increased Cost (Hardware/Software)
- Complexity (Setup/Management)
- Data Consistency Challenges

CAP THEOREM & USE CASES (System Design)

THE CAP THEOREM (Pick Two)



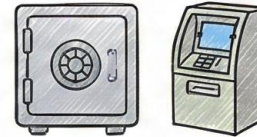
USE CASES & TRADE-OFFS

CA (Rarely Practical)



Traditional RDBMS in a single location/no partition.
Use Case: Specific internal applications where partitions are negligible.

CP (Consistency First)



Distributed Databases requiring strong consistency.
Use Case: Banking Systems, Financial Transactions (Must have accurate balance, can tolerate temporary unavailability).

AP (Availability First)



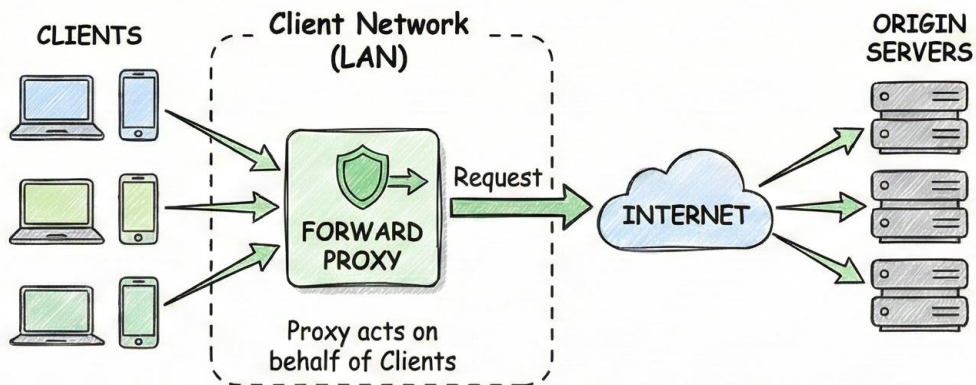
Systems where availability is crucial, eventual consistency is okay.
Use Case: Social Media Feeds, E-commerce Shopping Carts (Must always show content/allow adding to cart, data will sync eventually).

KEY TAKEAWAY:

- Design for Partition Tolerance (P) first
- Choose between Consistency (C) and Availability (A) based on business needs.
- 'Eventual Consistency' is often the practical choice for AP systems.

FORWARD VS REVERSE PROXY - DIFFERENCES & USE CASES (System Design)

FORWARD PROXY (Client's Advocate)



USE CASES



Content Filtering

Block access to specific sites (e.g., in corporate/school networks).



Anonymity

Hide client IP from origin servers.



Caching

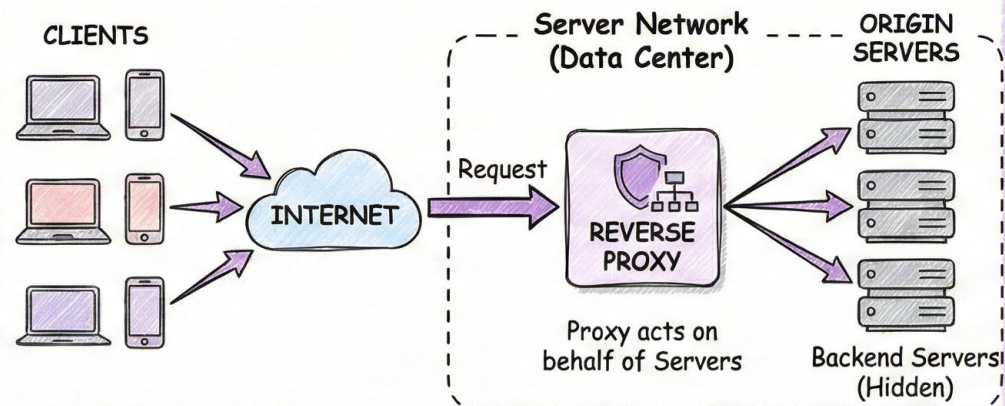
Cache frequently accessed resources to speed up access.



Geo-unblocking

Bypass regional restrictions.

REVERSE PROXY (Server's Advocate)



USE CASES



Load Balancing

Distribute incoming traffic across multiple servers.



Security / WAF

Protect servers from attacks (e.g., DDoS, SQL injection).



SSL Termination

Decrypt SSL traffic before sending to backend servers.



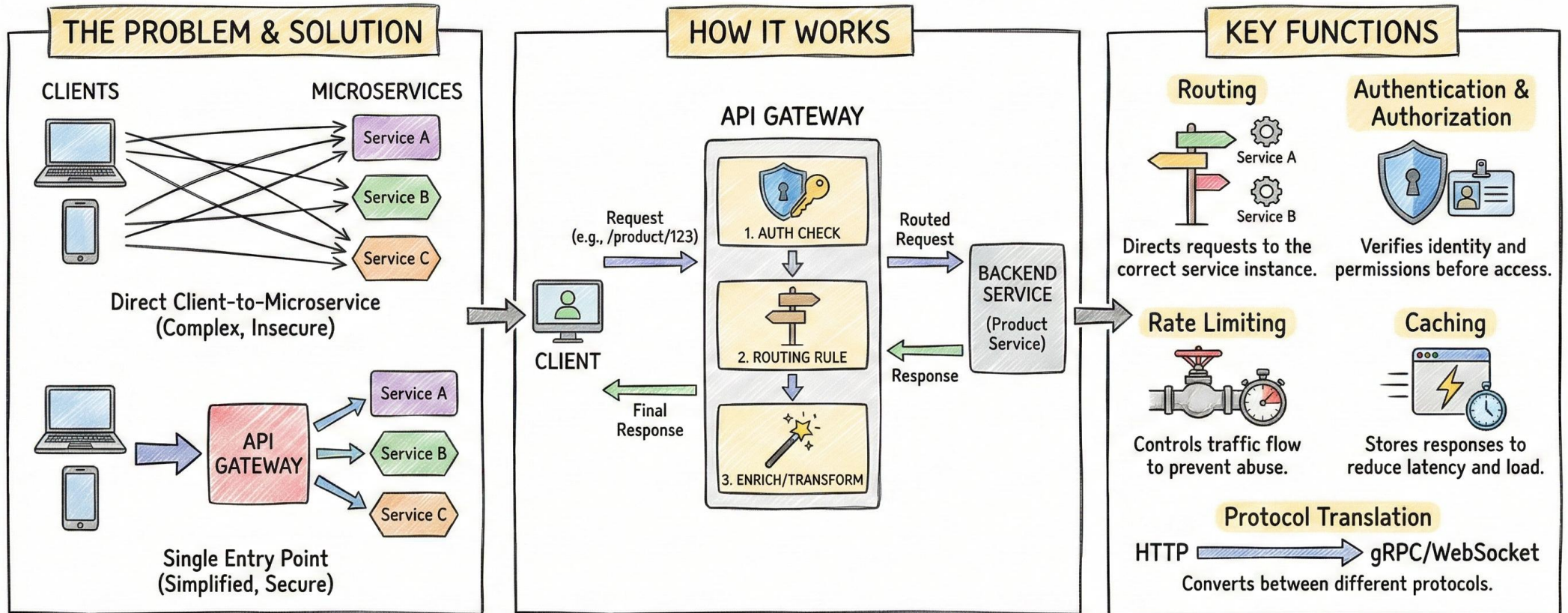
Caching & Compression

Cache static content and compress responses.

KEY TAKEAWAY:

- Forward Proxy: Hides Clients
- Reverse Proxy: Hides Servers
- Both enhance security, performance, and control.

API GATEWAY (System Design)

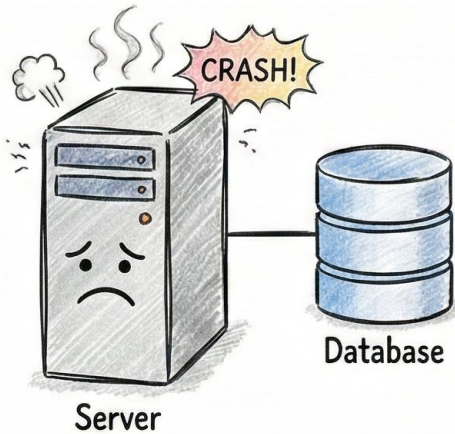


BENEFITS:

- Simplified Client Interface
- Improved Security Posture
- Centralized Management
- Reduced Backend Complexity

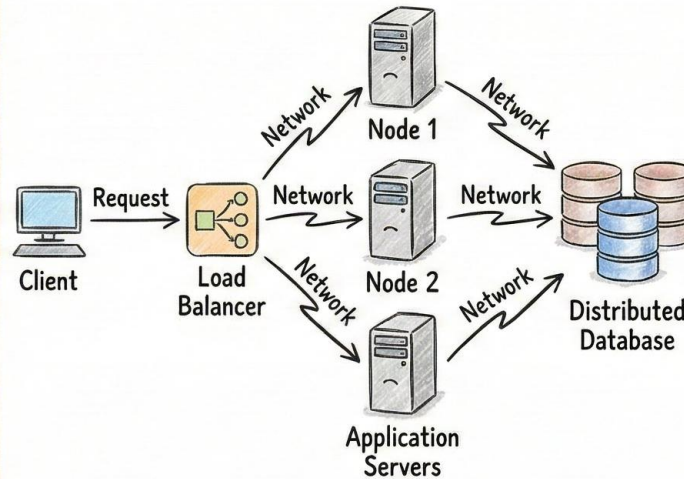
DISTRIBUTED SYSTEMS (System Design)

THE PROBLEM (The Monolith)



Problem: Limited resources, hard to scale, one failure crashes everything (Single Point of Failure).

THE SOLUTION (Distributed Systems)



Solution: Collection of independent computers that appear to its users as a single coherent system.

KEY CHARACTERISTICS & CHALLENGES

Characteristics

- ✓ Scalability (Horizontal - Add nodes easily)
- ✓ Reliability (Fault Tolerance - No single point of failure)
- ✓ Performance (Parallel Processing)
- ✓ Resource Sharing (Efficient use of hardware)

Challenges

- ⚠ Complexity (Harder to design, build, manage)
- ⚠ Network Issues (Latency, Partitions, Failures)
- ✗ Data Consistency (CAP Theorem trade-offs)
- ✗ Coordination (Synchronization, consensus is hard)

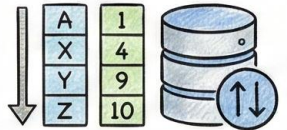
KEY TAKEAWAY: They offer massive scale and reliability but introduce significant complexity in coordination, consistency, and failure handling.

INDEXING IN DATABASES - USEFULNESS & CHALLENGES (System Design)

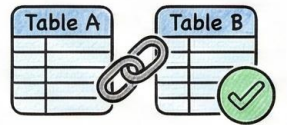
USEFULNESS



Faster Query Performance
Speedy data retrieval
(e.g., SELECT... WHERE...).



Efficient Sorting
Optimizes ORDER BY & GROUP BY clauses.



Improved Joins
Accelerates table linking
(e.g., JOIN operations).



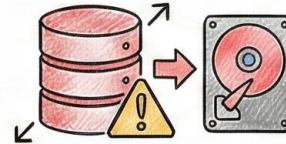
Enforced Uniqueness
Guarantees unique values
(e.g., PRIMARY KEY, UNIQUE constraints).



Optimized Aggregation
Speeds up functions like SUM, COUNT, AVG.



CHALLENGES



Increased Storage Space
Indexes consume additional disk space.



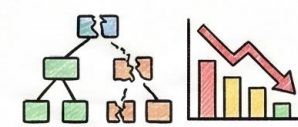
Slower Write Operations
INSERT, UPDATE, DELETE become slower.



Maintenance Overhead
Requires regular monitoring and rebuilding/reorganizing.



Complexity in Design
Choosing the right index type and columns is complex.



Index Fragmentation
Performance degrades over time, needs defragmentation.

KEY TAKEAWAY:

- Balance Read vs. Write Needs
- Index Selectively (Analyze Queries)
- Monitor & Maintain Regularly

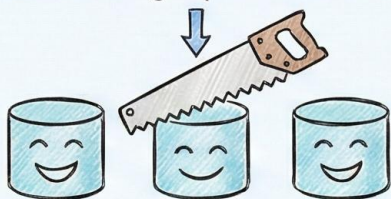
DATA PARTITIONING (System Design)

CONCEPTS & PROBLEM



Monolithic Database (Too Big!)

Problem: Difficult to scale, slow queries, single point of failure.



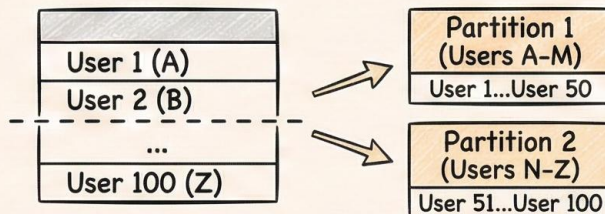
Partitions (Smaller & Manageable)



Solution: Divide data into independent, distinct parts.

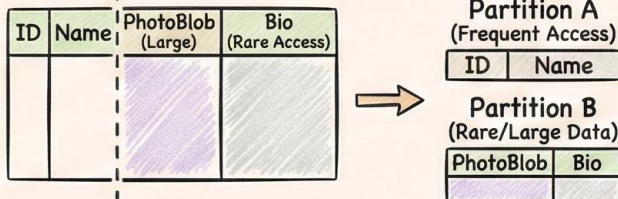
COMMON STRATEGIES (How to Split)

Horizontal Partitioning (Sharding)



Split by Rows (e.g., Range or Hash based).
Good for scaling data volume.

Vertical Partitioning



Split by Columns (e.g., by feature or access pattern). Optimizes for specific queries.

BENEFITS & CHALLENGES

- ✓ **Scalability** (Scale out easily)
- ✓ **Performance** (Faster queries on smaller data)
- ✓ **Availability** (Failure in one doesn't kill all)
- ✓ **Manageability** (Easier maintenance)
- ⚠ **Complexity** (More moving parts to manage)
- ✗ **Cross-Partition Joins** (Difficult & slow)
- ✗ **Rebalancing Data** (Can be complex to move data)

KEY TAKEAWAY: A fundamental technique for building large-scale systems, trading some complexity for significant gains in scalability and performance.

DESIGN A RATE LIMITER (System Design)

CONCEPTS

Goal: Control the rate of traffic sent by a client or service.



Requests per Second (RPS)

Limit rate over time.



Quota

Total allowed requests in a period.



Window Size

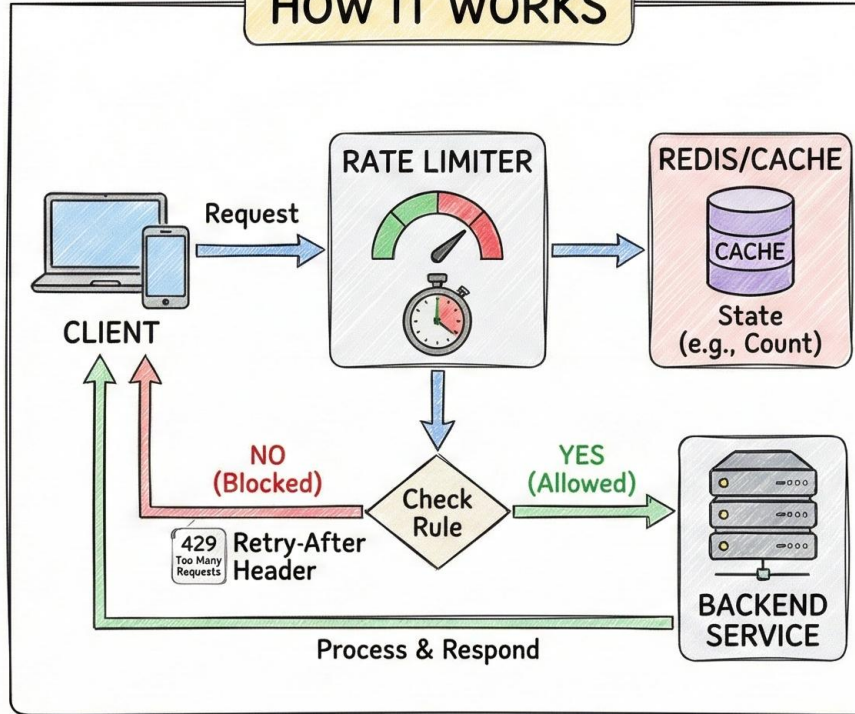
Timeframe for counting requests.



Throttling

Slowing down requests instead of blocking.

HOW IT WORKS



BENEFITS:

- Prevent Abuse/DoS Attacks
- Ensure Fairness Among Users
- Protect Backend Services
- Manage Costs & Resources

ALGORITHMS

Token Bucket



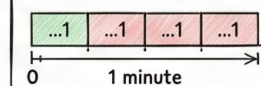
Tokens added at fixed rate; requests consume tokens.

Leaky Bucket



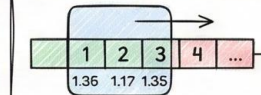
Requests enter a bucket; processed at a constant rate.

Fixed Window



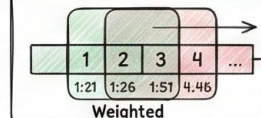
Count requests in a fixed time window (e.g., 1 min).

Sliding Window Log



Tracks timestamps; counts requests in the trailing window.

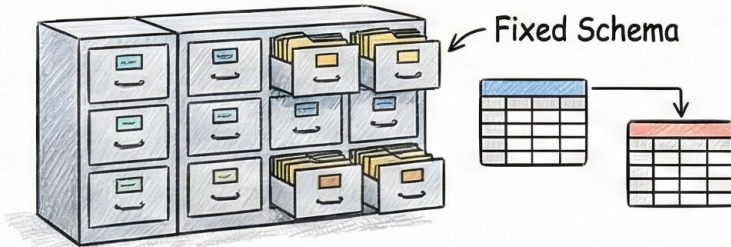
Sliding Window Counter



Hybrid approach; more accurate, combines fixed & sliding window concepts.

SQL VS NOSQL (Database Systems) - (System Design)

SQL (Relational DBs) - Structured & Rigid



ID	Name	Age

Data Structure
Tables, Rows, Columns
(Relational Model)



Schema
Fixed, Pre-defined Schema
(Changes are hard)



Scaling
Vertical Scaling
(Scale Up - Bigger Machine)



Query Language
Structured Query Language
(Powerful, complex queries)

Examples



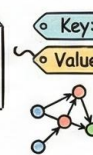
ORACLE



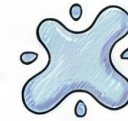
Best For
Complex queries,
Transactions (ACID),
Structured data.



NOSQL (Non-Relational DBs) - Flexible & Dynamic



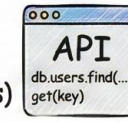
Data Structure
Document, Key-Value,
Graph, Column
(Diverse Models)



Schema
Dynamic, Schema-less
(Easy to change)



Scaling
Horizontal Scaling
(Scale Out - More Machines)



Query Language
API-based, focused methods
(Simpler, less powerful joins)

Examples



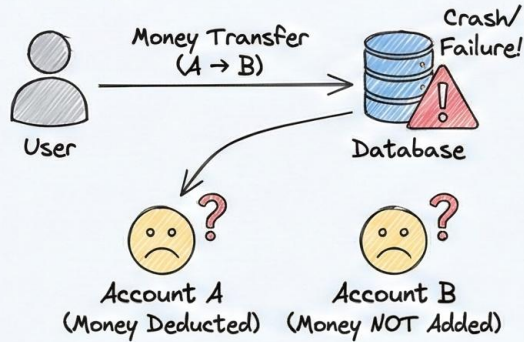
Best For
Rapid development, Massive
scale, Unstructured data,
High velocity.

KEY TAKEAWAY: Choose based on data structure, scalability needs, and query complexity.
SQL for complex queries & transactions; NoSQL for flexibility, speed & massive scale.

ACID PROPERTIES (Database Transactions) - (System Design)

CONCEPTS & PROBLEM (The Unreliable World)

System Without ACID



Problem: Without guarantees, data can become inconsistent or lost during failures.



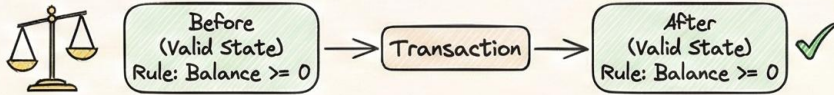
Solution: ACID - A set of properties that guarantee reliable processing of database transactions.

THE ACID PROPERTIES (The Guarantees)

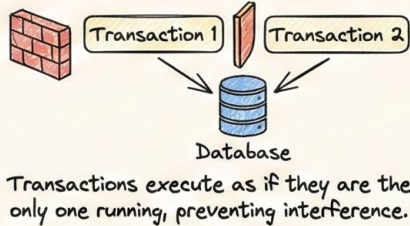
A - Atomicity ("All or Nothing")



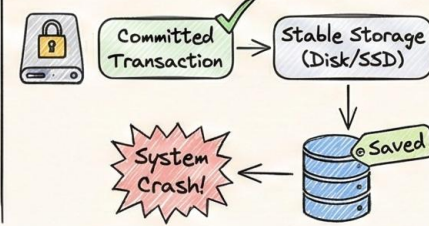
C - Consistency ("Valid State to Valid State")



I - Isolation ("Concurrent but Independent")



D - Durability ("Once Committed, It's Permanent")



BENEFITS & CHALLENGES

Benefits

- ✓ Data Integrity (Trustworthy)
- ✓ Reliability (Handles failures)
- ✓ Simplified Logic (Less error handling code)

Challenges

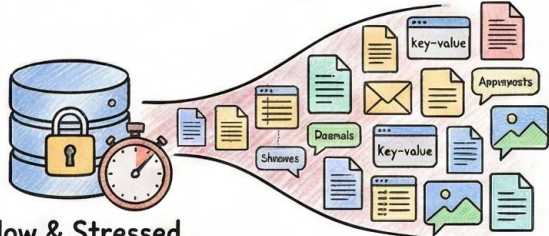
- ⚠ Performance Overhead (Slower due to locking/logging)
- ⚠ Scalability Issues (Harder to scale horizontally)
- ✗ Complexity (Implementation is complex)

KEY TAKEAWAY: ACID provides strong data guarantees, essential for critical systems (e.g., banking), but comes with performance trade-offs compared to BASE (NoSQL).

BASE PROPERTIES (NoSQL Databases) - (System Design)

CONCEPTS & PROBLEM (The Need for Flexibility)

Rigid ACID System



Slow & Stressed

Massive, Distributed Data
(High Velocity/Volume)



Problem: ACID's strictness limits scale and availability in massive, distributed systems.



Solution: BASE - An alternative model for distributed systems prioritizing availability and scale over immediate consistency.

THE BASE PROPERTIES (The Trade-offs)

→ BA - Basically Available ("Always Responsive, Mostly")



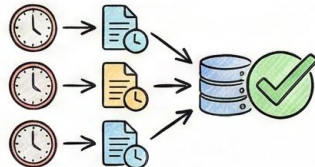
System guarantees availability for most requests, but maybe not all data is reachable at all times.

→ S - Soft State ("State Can Change, Even Without Input")



State of the system may change over time, due to internal processes (like replication), not just user updates.

→ E - Eventual Consistency ("Consistent, Eventually")



System will become consistent over time, given that updates stop. Inconsistency is temporary.

BENEFITS & CHALLENGES (vs. ACID)

→ Benefits

- ✓ High Scalability (Scale out easily)
- ✓ High Availability (Resilient to failures)
- ✓ Flexible Schema (Adapts to change)
- ✓ Better Performance (Lower latency)

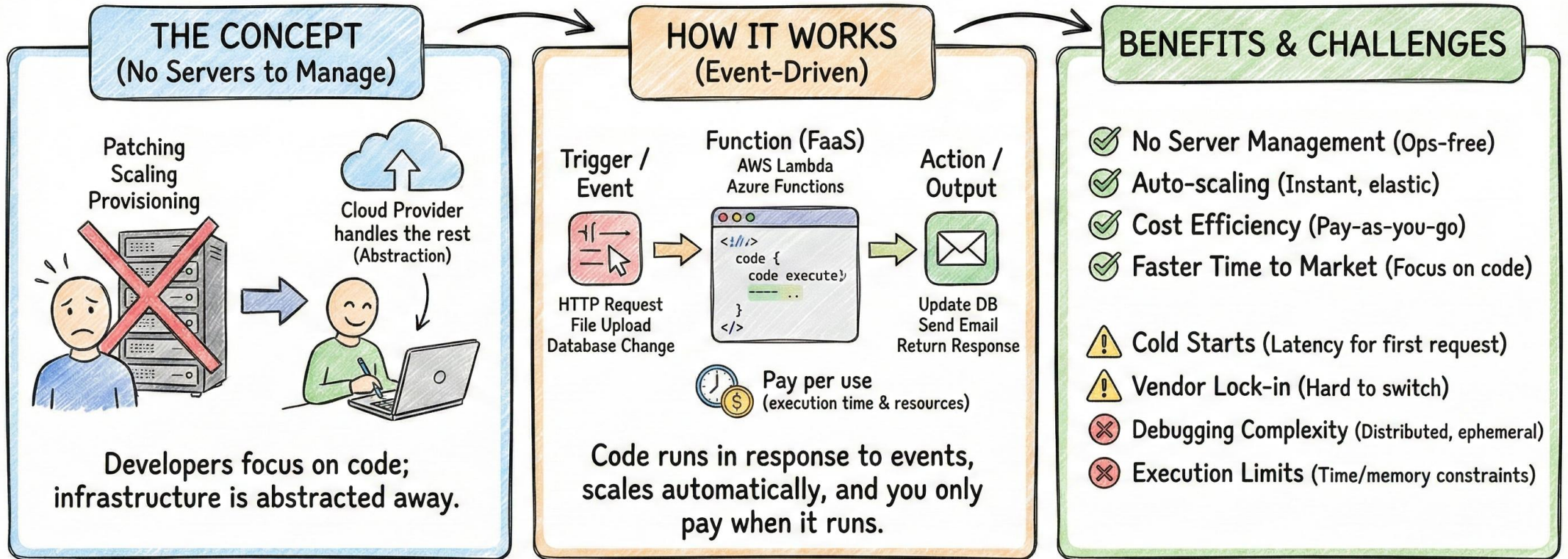
→ Challenges

- ⚠ Data Inconsistency (Temporary versions)
- ⚠ Complex App Logic (Handling conflicts)
- ✗ No Strong Guarantees (Trade-off)
- ✗ Not for Critical Transactions (e.g., Banking)

KEY TAKEAWAY: BASE trades immediate consistency for higher availability and scalability, making it suitable for large-scale, distributed web applications and big data (AP in CAP).

SERVERLESS COMPUTING

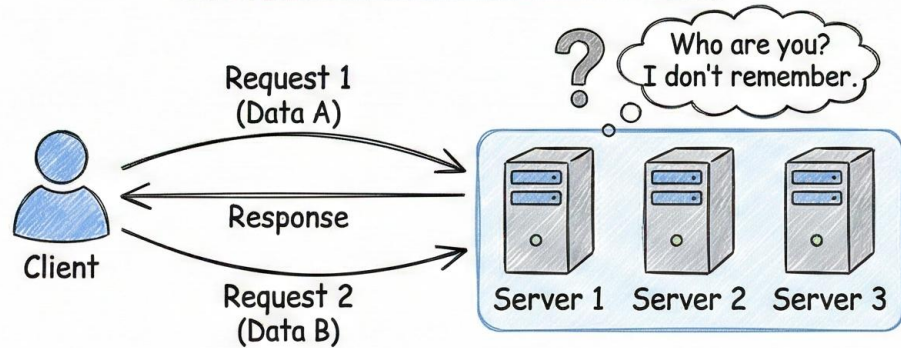
(System Design)



KEY TAKEAWAY: Focus on writing code, not managing infrastructure.
Ideal for event-driven, scalable applications with variable workloads.

STATELESS VS STATEFUL (System Design)

STATELESS (No Memory)



Each request is independent. Server doesn't save client data between requests.

Pros:



Scalability
(Easy to add servers, any server can handle any request)



Fault Tolerance
(If a server fails, another takes over easily)

Cons:



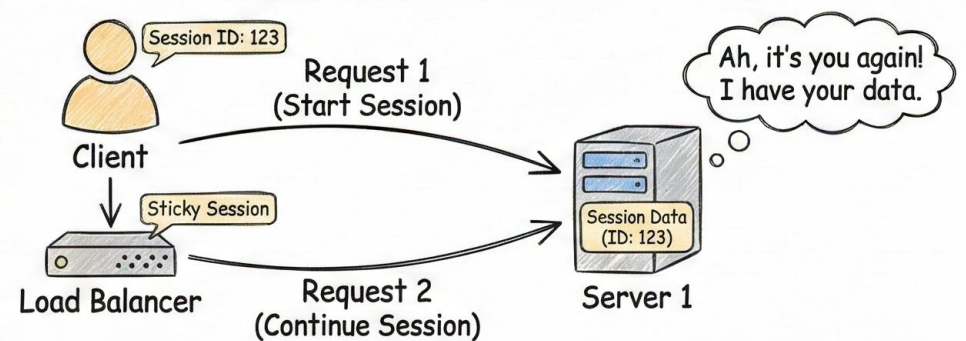
Data Overhead
(Client must send all necessary data with every request)



Complexity
(Managing state externally, e.g., in a DB or cache)

Best for: REST APIs, CDN, Microservices (often).

STATEFUL (Has Memory)

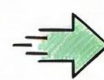


Server retains client data (state) across multiple requests.

Pros:



Simpler Logic
(State is available locally on the server)



Performance
(Less data transfer per request)

Cons:



Scalability Issues
(Sticky sessions make scaling harder)



Fault Tolerance
(If server fails, session data is lost)

Best for: Online Gaming, Databases, FTP, Real-time apps.

KEY DIFFERENCES:

Aspect

Focus
Data Handling
Coupling

Stateless

Independent Requests
Client sends data
Tight

Scalability

High (Easy)
High (Easy)
Loose

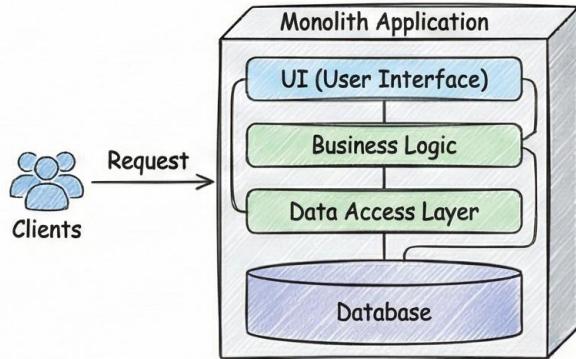
Stateful

Client Session
Server keeps data
Limited (Harder)

MONOLITHIC VS MICROSERVICES ARCHITECTURE

(System Design)

MONOLITHIC ARCHITECTURE (The Big Block)



Single, unified unit. All components bundled together.

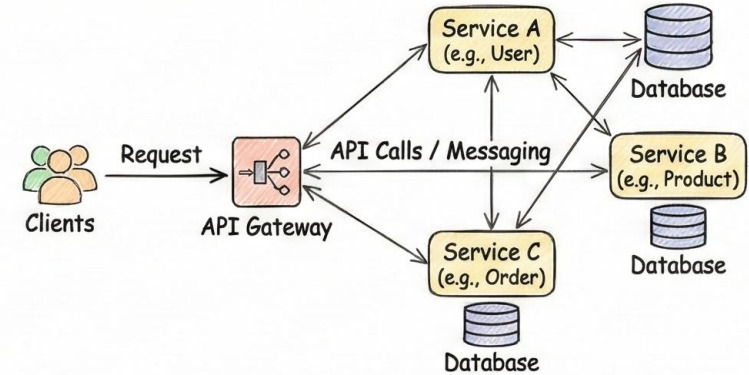
Pros:

- Simplicity**
(Easier to develop & test initially)
- Simple Deployment**
(One file to deploy)

Cons:

- Limited Scalability**
(Scale entire app, even if only one part needs it)
- Complexity over Time**
(Becomes hard to understand & maintain)
- Single Point of Failure**
(A bug in one part can bring down the whole system)
- Technology Lock-in**
(Hard to change tech stack)

MICROSERVICES ARCHITECTURE (Small, Connected Blocks)



Collection of small, autonomous services communicating via APIs.

Pros:

- High Scalability**
(Scale individual services independently)
- Fault Isolation**
(Failure in one service doesn't crash the whole system)
- Technology Diversity**
(Use different stacks for different services)
- Independent Deployment**
(Faster, more frequent updates)

Cons:

- Increased Complexity**
(Distributed system challenges)
- Data Consistency**
(Harder to maintain across services)
- Network Latency**
(Communication between services adds time)

KEY DIFFERENCES & TRADE-OFFS:

Aspect

Scalability
Development
Complexity
Data Management

Monolithic

Limited (Scale All)
Simpler Initial
Lower (Initially)
Centralized (Simpler)

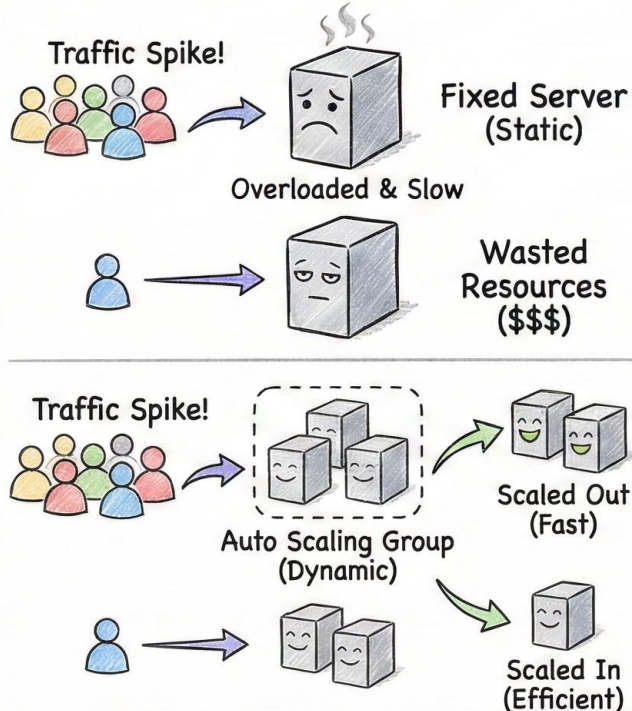
Microservices

High (Scale Independent)
Complex (Distributed)
Higher (Operational)
Decentralized (Complex)

AUTOSCALING (System Design)

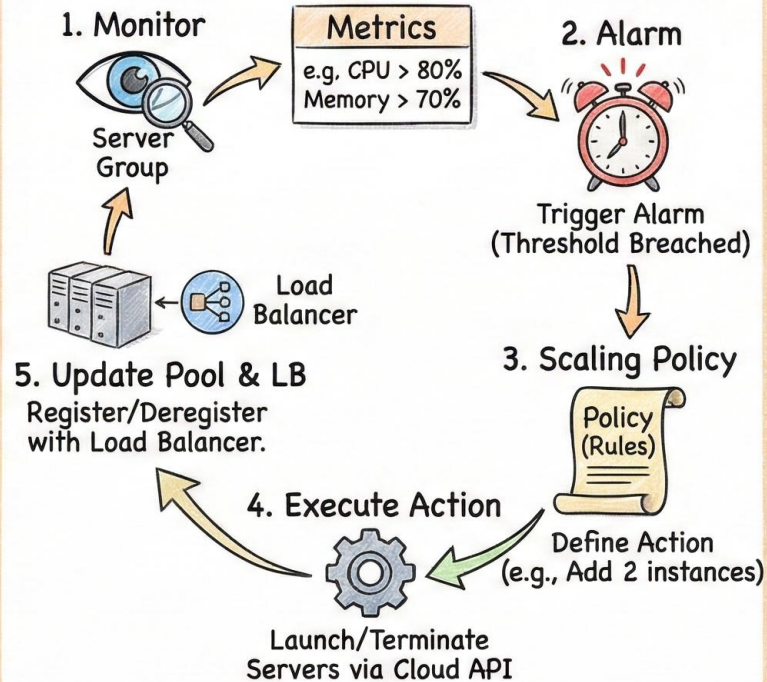
THE CONCEPT

(Dynamic Resource Adjustment)



HOW IT WORKS

(The Scaling Loop)



BENEFITS & STRATEGIES

- ✓ **Cost Savings**
(Pay only for what you use)
- ✓ **High Availability**
(Handles spikes & failures)
- ✓ **Consistent Performance**
(No overload)
- ✓ **Reduced Management**
(Automated)

Scaling Strategies

- Dynamic/Reactive**
Scale based on real-time metrics (e.g., CPU).
- Predictive**
Scale based on historical patterns & ML.
- Scheduled**
Scale at specific times (e.g., before a sale).

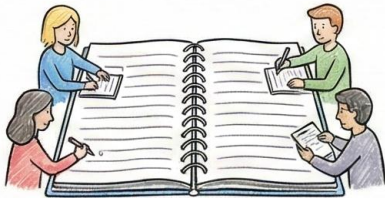
KEY TAKEAWAY: Automatically adjusts computing resources to match demand, ensuring performance and cost-efficiency in dynamic environments.

STRONG VS EVENTUAL CONSISTENCY (System Design)

STRONG CONSISTENCY (Strict & Immediate)



Same View at Same Time



All nodes see the same data at all times.



Updates are immediate and block until acknowledged by all replicas.



Higher Latency (Waiting for synchronization).



Lower Availability (Prioritizes consistency over uptime - CP in CAP).



Data View

All nodes see the same data at all times.



Complex to implement & manage (e.g., Distributed Transactions).



Updates are immediate and block until acknowledged by all replicas.



Examples

Critical data, bhnse Ticker (e.g., Financial Transactions).



RDBMS (ACID)



Banking Systems



Stock Market Ticker



Best For

Critical data, Financial transactions, Systems requiring absolute accuracy.

VS



EVENTUAL CONSISTENCY (Flexible & Delayed)



Different Views Temporarily



Nodes may see different data versions temporarily.



Updates propagate asynchronously; system becomes consistent over time.



Lower Latency (Non-blocking, fast responses).



Higher Availability (Prioritizes uptime over immediate consistency - AP in CAP).



Data View

Nodes may see different data versions temporarily.



Simpler to implement & scale (e.g., Gossip Protocols).



Updates propagate asynchronously; system becomes consistent over time.



Examples

Enporates to implement & ehove (e.g., Gossip Protocols).



DNS



Social Media Feeds



CDN



NoSQL (BASE)



Best For

High-traffic web apps, Content delivery, Systems where temporary inconsistency is acceptable.

KEY TAKEAWAY: Choose based on the trade-off between immediate data accuracy (Strong) and system performance/availability (Eventual). Strong for critical data; Eventual for speed and scale.

PUB/SUB MODEL (Publish-Subscribe) - (System Design)

CONCEPTS & ROLES



Publisher (Sender)

Creates & sends messages.
Doesn't know who receives them.



Subscriber (Receiver)

Expresses interest in messages.
Doesn't know who sends them.



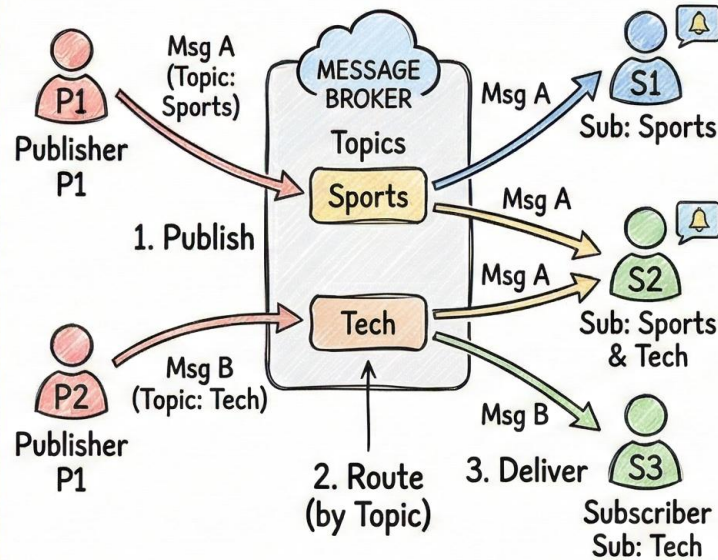
Topic / Channel (Intermediary)

Named channel where messages are sent and received.



Decoupling: Senders and receivers are independent.

HOW IT WORKS (The Flow)



BENEFITS & USE CASES

Benefits:

- Loose Coupling (Independent scaling/evolution)
- Scalability (Handles many pubs/subs)
- Asynchronous (Non-blocking)
- Flexibility (Dynamic subscriptions)

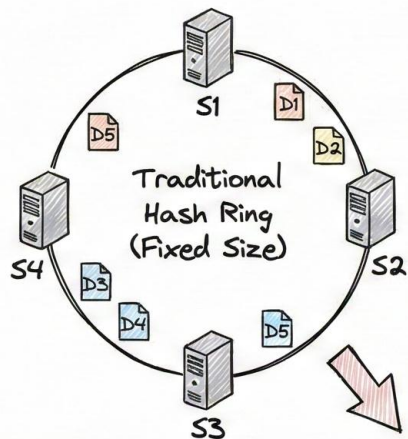
Use Cases:

- Event Notifications (e.g., social media updates)
- Chat Applications (Group messages)
- IoT Data Feeds (Sensor readings)
- Distributed Logging & Analytics

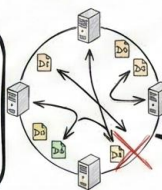
KEY TAKEAWAY: Decoupled, asynchronous communication pattern for building scalable and flexible distributed systems.

CONSISTENT HASHING (System Design)

CONCEPTS & PROBLEM

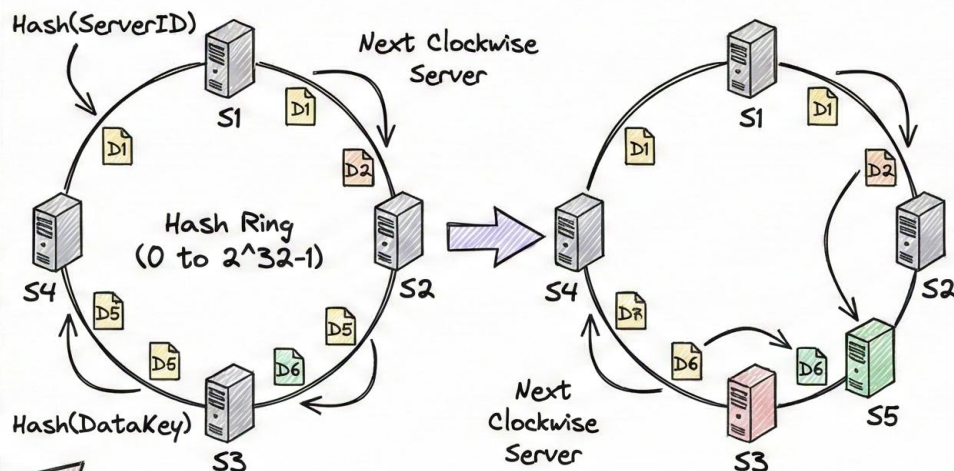


Problem: Adding/removing a server requires remapping MOST data (e.g., $\text{Hash}(\text{Key}) \% N$).



HOW IT WORKS

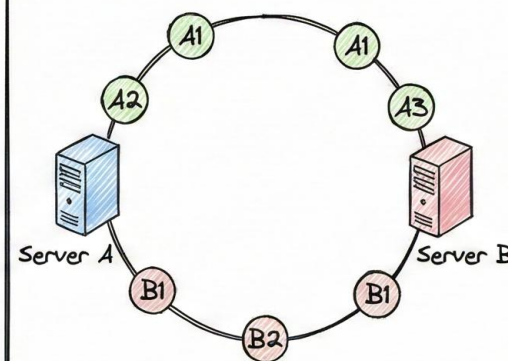
(The Ring)



Only data between S2 and S5 is remapped to S5. Minimal data movement.

VIRTUAL NODES

(Balancing)

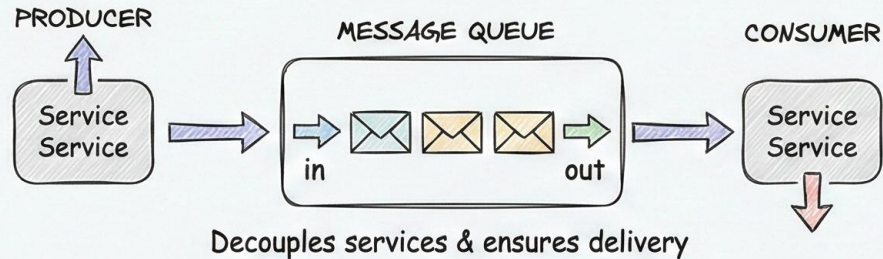


BENEFITS & USE CASES:

- Minimal Rebalancing
- Seamless Scalability
- Hotspot Mitigation
- Distributed Caching (e.g., Dynamo, Memcached)
- Load Balancing
- Data Partitioning

MESSAGE QUEUES & CDN (Content Delivery Network) - (System Design)

MESSAGE QUEUES (Asynchronous Messaging)



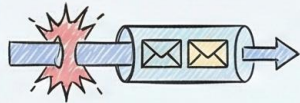
USE CASES



- **Asynchronous Processing:** Offload time-consuming tasks (e.g., email sending, report generation).

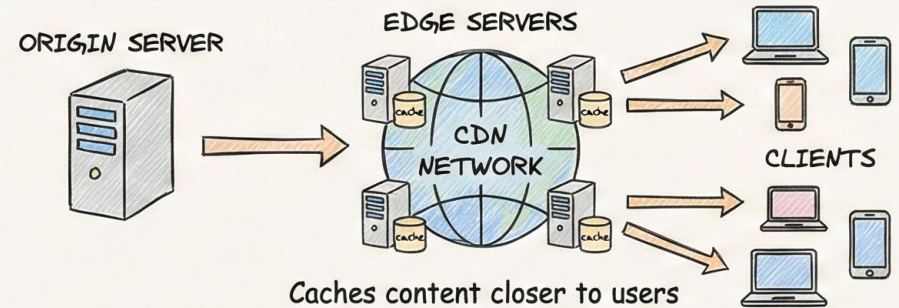


- **Decoupling:** Services don't need to know about each other.



- **Reliability/Buffering:** Handle traffic spikes and ensure data isn't lost if a consumer is down.

CDN (Global Caching)



USE CASES



- **Faster Content Delivery:** Deliver static assets (images, videos, CSS, JS) quickly to end-users.



- **Reduced Server Load:** Offload traffic from the origin server, preventing overload.



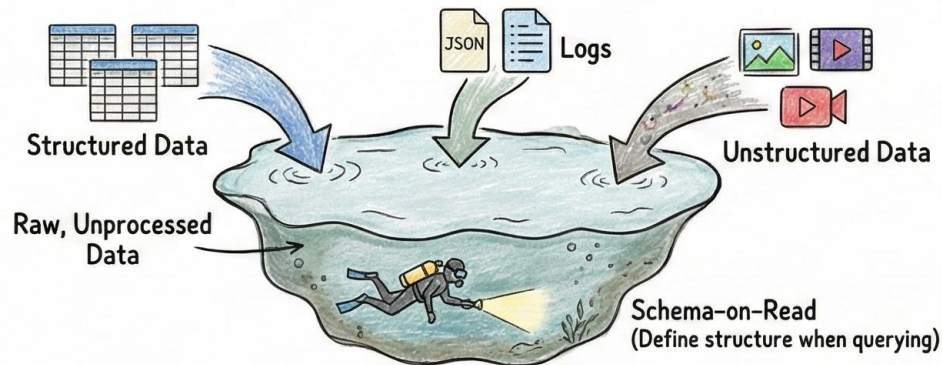
- **Improved Availability:** Handle high traffic & mitigate DDoS attacks through distributed infrastructure.

KEY TAKEAWAY:

- **Message Queues:** Decouple & Handle Spikes (Backend)
- **CDN:** Accelerate Content Delivery (Frontend/Global)
- Both improve scalability and performance.

DATA LAKE VS DATA WAREHOUSE (System Design)

DATA LAKE (The Reservoir) - Raw & Flexible



Data Types
Stores ALL data types
(Structured, Semi-structured,
Unstructured).

**Data Scientists,
Data Engineers, Analysts**
(Exploration, ML).

AWS S3 **Azure Data
Lake Storage** **Google Cloud
Storage**



ELT
(Extract, Load, Transform)
- Load first, transform later.

Low Cost Storage
(Commodity hardware,
object storage).

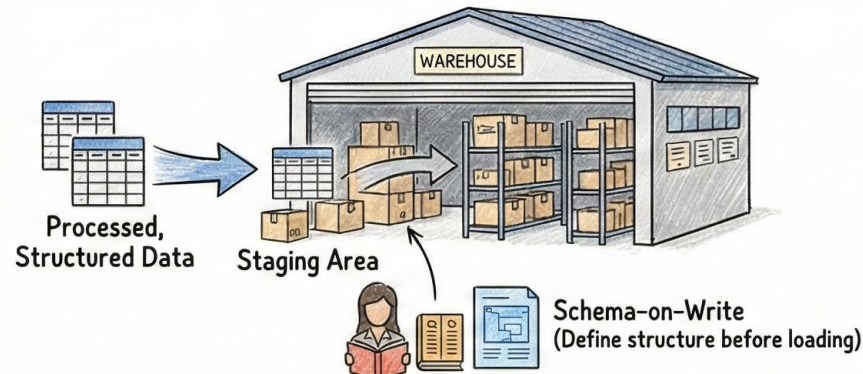


Best For:

Data exploration,
Machine Learning, storing
vast amounts of raw data,
unknown future queries.



DATA WAREHOUSE (The Structured Store) - Curated & Fast



**Stores mainly
Structured data**
(Relational, modeled).

**Business Analysts,
BI Professionals**
(Reporting, Dashboards).

Snowflake **Amazon
Redshift** **Google
BigQuery**



ETL
(Extract, Transform, Load)
- Transform before loading.

Higher Cost Storage
(Optimized, high-performance
storage).



Best For:

Business Intelligence (BI),
reporting, complex analytical
queries, historical analysis.

KEY TAKEAWAY: Choose based on data type and use case. Data Lake for flexibility and raw data exploration;
Data Warehouse for structured, high-performance analytics and reporting.

BIG DATA (System Design)

